TERMINAL APPLICATION · C PROGRAMMING PROJECT

MEMORY FLASH GAME

A COLOR SEQUENCE MEMORY CHALLENGE | DEVELOPED IN C

C LANGUAGE · LINUX TERMINAL · COGNITIVE UX · MODULAR DESIGN

PROJECT OVERVIEW

Memory Flash Game is a terminal-based C program that displays a growing color sequence — **Red**, **Green**, **Blue**, **Yellow** — and challenges the player to repeat it from memory. Each round adds one more color. A **persistent high score** is tracked across replays.

GAMEPLAY MECHANICS

1. **INITIALIZE**
   Seed RNG, reset sequence and level counter
2. **SHOW COLOR**
   New random color flashes for 2 seconds
3. **PLAYER INPUT**
   User re-enters the full sequence (1–4)
4. **VALIDATE**
   Each input checked; mismatch ends the game

COLOR ENCODING SYSTEM (ANSI)

1 / RED	#FF3B30	\033[1;41m
2 / GREEN	#30D158	\033[1;42m
3 / BLUE	#0A84FF	\033[1;44m
4 / YELLOW	#FFD60A	\033[1;43m

KEY FEATURES

- **Progressive Difficulty**
  One new color added each round, steadily increasing challenge
- **Persistent High Score**
  Best score stored and shown on every game-over screen
- **Input Validation**
  Values outside 1–4 are rejected with a clear invalid-input error
- **Replay System**
  Player can restart immediately after any game without re-launching
- **ANSI Color Blocks**
  Full background ANSI colors with emoji overlays for vivid display
- **Score Tracking**
  currentScore compared to highScore after every game-over event

VALIDATION & SCORING LOGIC

- **Loop — Collect Input**
  for loop: i = 0 to level-1 — collect each color input from the player
- **Range Check — Invalid Input**
  If input < 1 or > 4 — trigger INVALID INPUT and end the game immediately
- **Sequence Check — Compare**
  Compare playerInput[i] vs correctSequence[i] for each position in turn
- **Mismatch — Wrong Sequence**
  Any mismatch triggers WRONG SEQUENCE and ends the current game round
- **All Match — Advance Level**
  All inputs correct — CORRECT result, level++ and proceed to the next round
- **Game Over — Update High Score**
  On game over, update highScore if currentScore exceeds the stored best

TECHNICAL SPECIFICATIONS

01	Language: `C`	02	Compiler: `GCC / MinGW`
03	Platform: `Linux / Win`	04	Libraries: `stdio · stdlib`
05	Colors: `ANSI Escape`	06	Input: `scanf()`
07	RNG: `srand(time(0)) + rand() % 4 + 1`		

time.h · unistd.h · stdlib.h · stdio.h

DEVELOPED BY

01 Vineet Shirode
02 Rajvi Thuse
03 Snigdha Dutt
04 Arohi Govindu